NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SC4000 MACHINE LEARNING

Project report

Semester 1, AY 2025/2026

Group 17

Group Members

U2222356E	Du Camille Abigail De Jesus	c220163@e.ntu.edu.sg
U2223241C	Agarwal Dhruv	dhruv018@e.ntu.edu.sg
U2223741K	Kavipooja Selvakumar	kavipooj001@e.ntu.edu.sg
U2223431K	Caren Tan Xin Yao	ctan270@e.ntu.edu.sg
U2222769H	Yao Yuzhu	c220189@e.ntu.edu.sg

Instructor

Ke Yiping, Kelly & Liu Siyuan

Date

14 November 2025

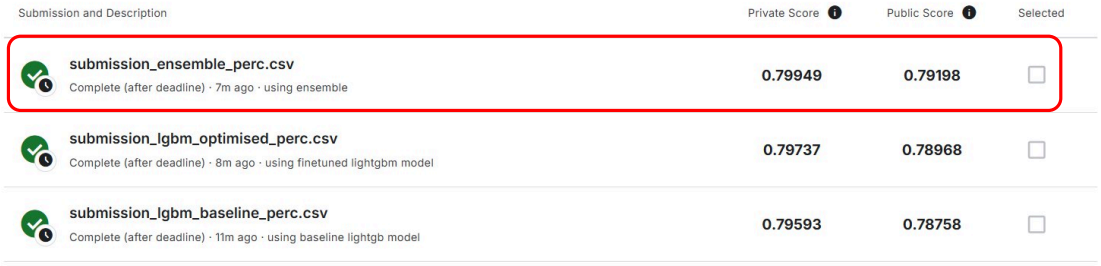
Table of Contents

1. Leaderboard Results

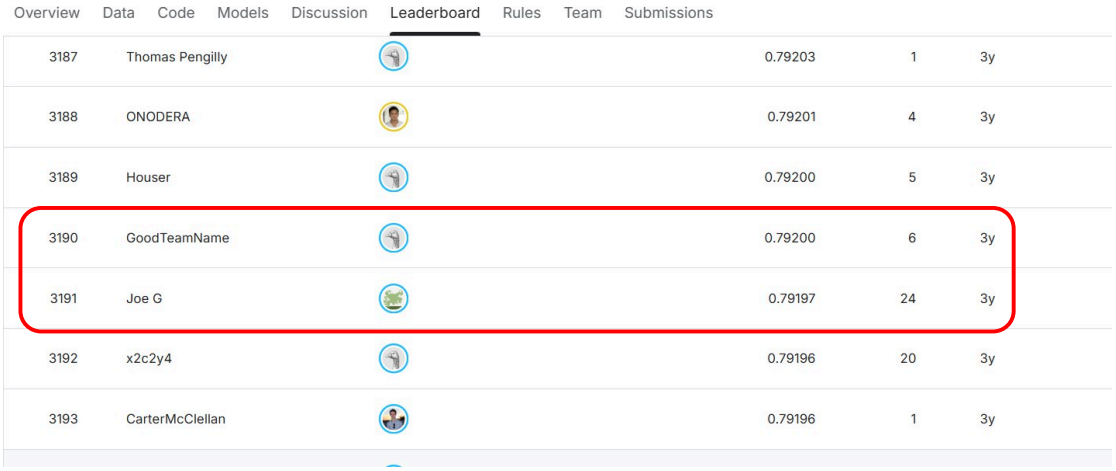
Summary of model performance:

Model	Baseline AMEX	Tuned AMEX	Improvement
LightGBM + XGBoost + CatBoost (all optimised)	-	0.796716	+0.00241(from LightGBM baseline model), +0.00075 (from LightGBM optimised model)
LightGBM	0.794310	0.795962	+0.00165
XGBoost	0.792682	0.795224	+0.00254
CatBoost	0.792617	0.794574	+0.00196

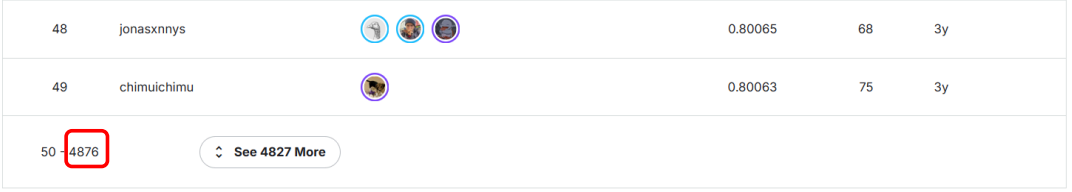
As shown in the table above, since the ensemble is the best, we chose to use it to calculate our ranking position which results in a public score of 0.79198.



By comparing the scores on the public leaderboard, position 3190 has a score of 0.79200 while position 3191 has a score of 0.79197. This means that our score of 0.79198 will replace position 3191.



As the total number of positions on the leaderboard is 4876 originally, our score places us in the top 3191/(4876+1) = 65.43%.



2. Competition Description

2.1. Project Outline and Aim

The objective of this competition is to identify customers likely to default on loans using the American Express (AMEX) prediction dataset. Accurate prediction of defaults helps banks lend responsibly and reduce financial risk.

We evaluate three gradient-boosting models to predict customer default: LightGBM, XGBoost, and CatBoost. The target variable is binary, where 1 indicates a default and 0 indicates no default.

2.2. Dataset Description

The dataset contains monthly customer-level financial and behavioral features (D_*, S_*, P_*, B_*, R_*). This is provided in the following files:

1. train_label.csv: contains the Customer_ID and the target variable default
2. train_data.ftr: contains the customer-level features (D_*, S_*, P_*, B_*, R_*) for each observation.
3. test_data.ftr: has the same structure as train_data.ftr

The two datasets train_label.csv and train_data.ftr are merged on Customer_ID to create the final training set for model development.
After model training and fine-tuning, predictions were generated for test_data.ftr and submitted to Kaggle to obtain leaderboard rankings.

2.3. Evaluation Metric

The competition uses the AMEX Default Score, a custom metric that evaluates how well the model ranks customers by risk of default. It combines:
1. Normalized Gini coefficient which measures overall ranking quality
2. Default rate captured at the top 4% which measures how many actual defaulters appear in the highest-risk group

The formula: AMEX Score = 0.5(G+D)

A higher AMEX score indicates better model performance in correctly identifying and ranking customers with higher default risk.

3. Exploratory Data Analysis

EDA was conducted before the model training to understand the dataset structure, identify inconsistencies and eventually guide feature engineering decisions.

3.1. Data Cleaning

The initial dataset contained over 5.5 million rows across 190 features. We used the Feather format (train_data.ftr) to efficiently load the data due to memory constraints. Below are the data cleaning steps:

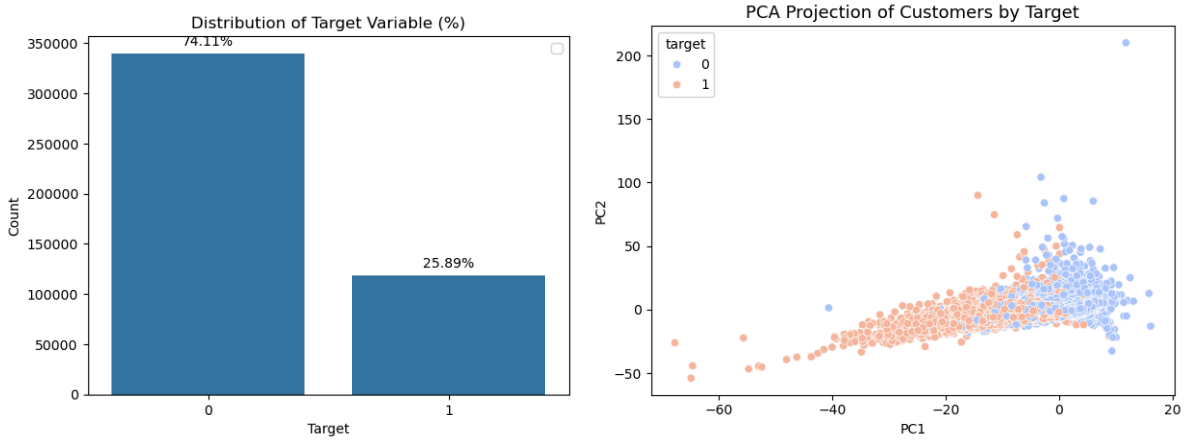
1) Aggregation
   First, we converted the time-series-like data (train_data.ftr) into one customer_id row per customer, by performing aggregation on both numeric and categorical columns. For the S_2 column, we picked the most recent date. Using the aggregation functions of [std, mean, min, max and last], we created sub-columns for each of the numeric columns. We then aggregated the categorical columns by using the [last] function to get the most recent categorical value. At this stage, we did not perform any imputations of NaN values or remove any columns with mostly NaNs, as we left that for the feature engineering and pre-processing stage.

2) Merging train_data.ftr and train_labels.csv
   After aggregation, we were left with 458913 rows, reduced from the original 5531451. This matched the number of rows in train_labels.csv We then merged train_data.ftr with train_labels.csv on the customer_id column, creating a unified training dataset, merged_and_aggregated_train_df.

3.2. Data Exploration and Analysis

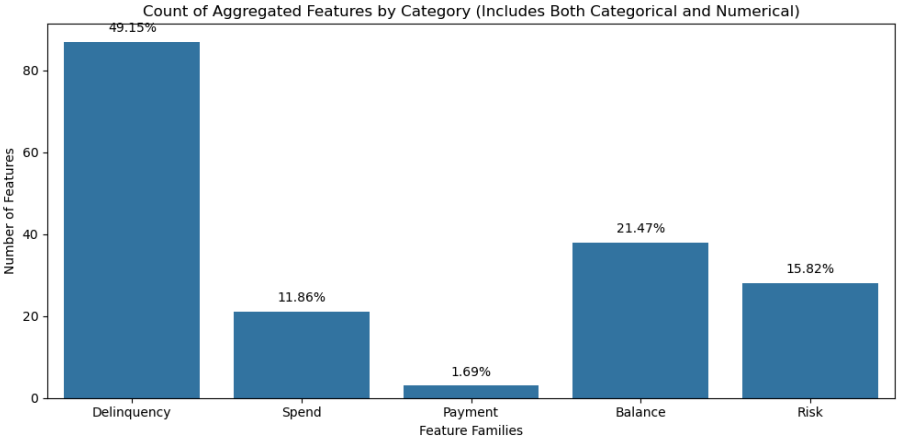
3.2.1. Visualisation of target variable distribution in the train data

Fig 1 shows that the 74.11% of customers in the merged_train_df have not defaulted. According to the PCA Projection of the binary data, we found that a simple linear boundary is insufficient as it shows partial overlap between defaulting and non-defaulting customers (ie. no distinct separation between the two). This highlights the non-linear relationship of the data, which will be explored later in our tree-based training models LightGBM, CatBoost and XGBoost in our project's training phase.



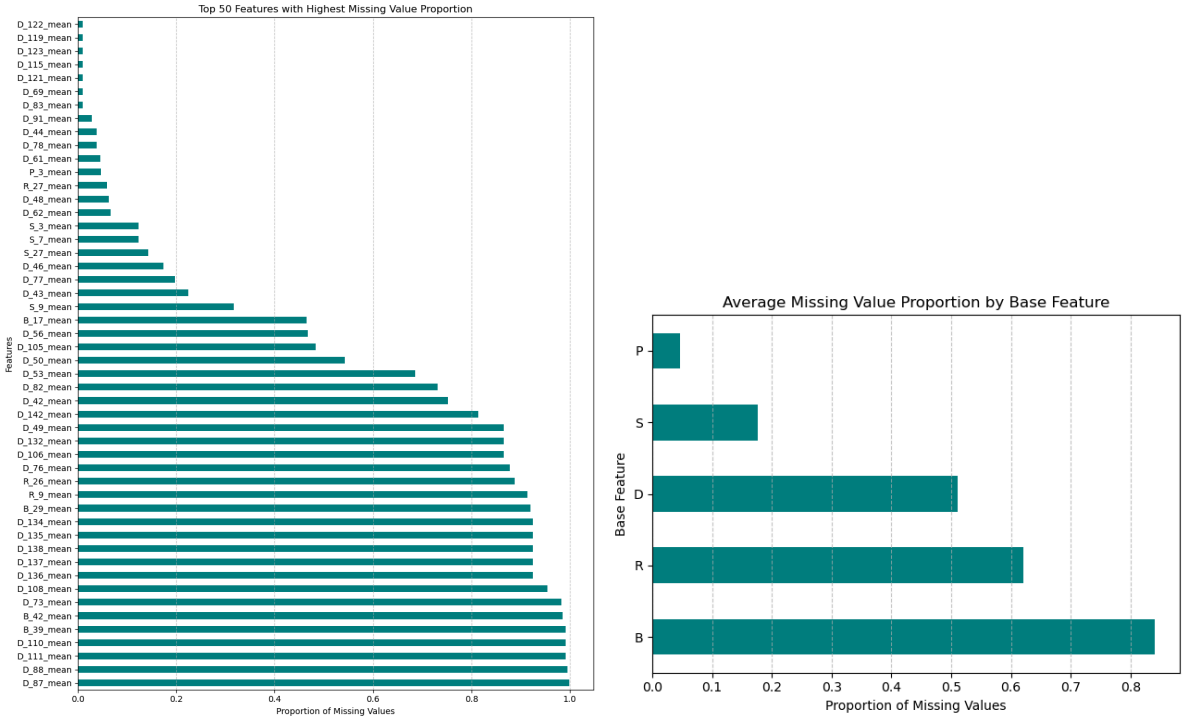
3.2.2. Visualisation of each feature's distribution

Among the 5 data types, it can be observed that Deliquency has the highest presence in merged_train_df.



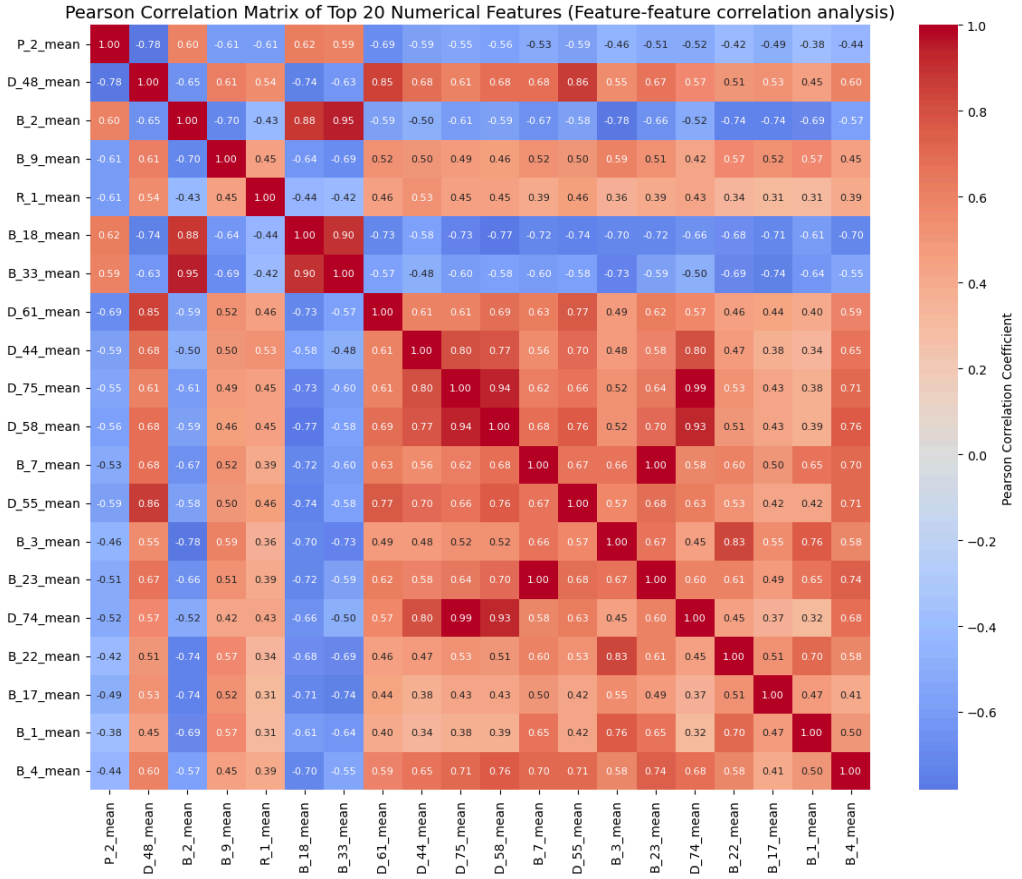
3.2.3. Null value Counting

In Fig 4, we observed that columns like D88_mean and D87_mean have missing value proportions close to 100%, which could indicate that these features may not be suitable for usage in our models. Among each feature type, we also observed that the Balance features had the most missing values among all features. We explore this further later on in the training portion of our report.



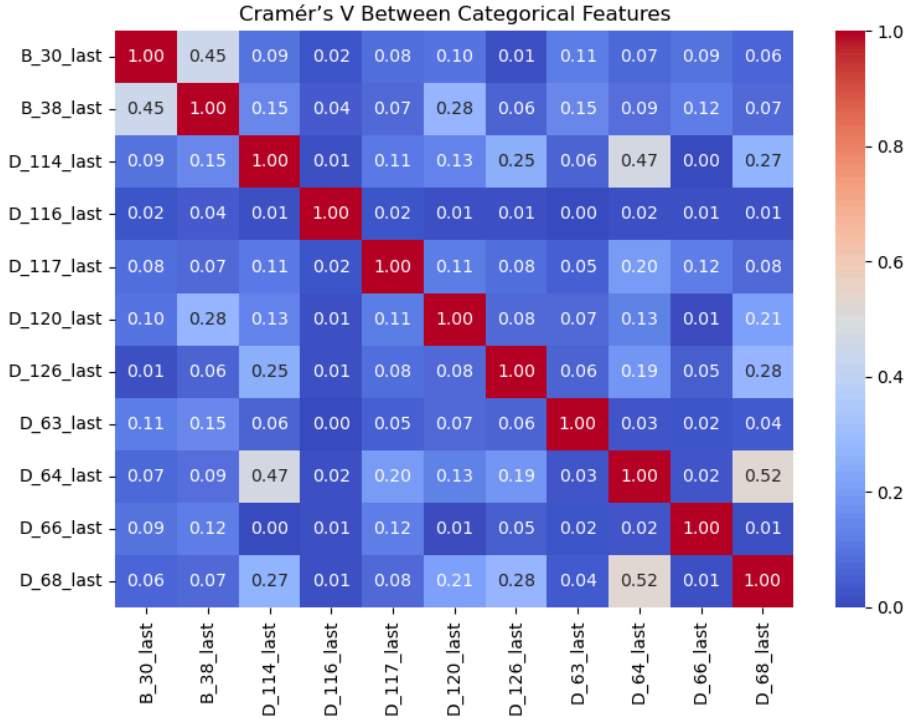
3.2.4. Correlation matrix for numerical features

Among the top 20 numerical features with highest correlation to the target, we plotted a Pearson Correlation Matrix and found that certain features had significant associations between them. For example, in the correlation matrix we can see a particularly strong correlation between D74_mean and D75_mean (0.93), as well as between B2_mean and B33_mean (0.95).



3.2.5. Correlation matrix for ordinal features

Similarly for the categorical features, we used Cramer's V and found generally weaker associations between variables instead. The strongest correlation score was 0.52, between D68_last and D64_last.



3.3 Feature Engineering and Preprocessing

3.3.1 One-hot Encoding

The dataset contains both numerical and categorical features, with high correlation among numerical data and lower correlation among categorical data.

While XGBoost, LightGBM, and CatBoost can handle categorical data natively, their approaches differ, and relying on native handling alone can lead to inconsistencies across models. To provide a uniform representation of key categorical variables and ensure fair comparison, we applied one-hot encoding selectively.

To limit feature dimensionality and memory usage, we only applied to the top 3 most influential categorical columns. An influential column is determined by the difference between the largest and smallest category-wise mean of the target. A higher range indicates greater variation and stronger impact on the target, so columns with larger ranges were considered more predictive.

	Influence Range
B_38_last	0.668833
B_30_last	0.482680
D_116_last	0.449579

Each of these columns is converted into multiple binary features that replace the original columns for model training.

3.3.2 Handling NaN values

XGBoost / LightGBM**:** Automatically handle NaNs during tree splits by sending missing values left or right to maximize gain. However, when categorical features are encoded as integers, NaNs may be misinterpreted as a valid category, so preprocessing (imputation) is needed.
CatBoost: Natively treats missing values as a distinct category, preserving the fact that a value was originally missing.

To maintain a consistent pre-processing pipeline across all 3 models, we applied the same NaN-handling strategy uniformly. Below describes our preprocessing approach:

- Categorical columns: Replace NaNs with the most frequent value (mode).
- Numeric columns: Replace NaNs with the column mean

This ensured that all models received the same cleaned dataset and avoided discrepancies in how missing values were interpreted.

4. Proposed Solution

4.1. Model choice and methodology

Our proposed models focus on utilizing gradient boosting ensemble approaches to tackle the issue of credit default prediction in a large, structured financial dataset. In financial risk modelling, tree-based ensemble models are chosen for their proven capacity to handle complicated or a variety of data types, capture intricate non-linear relationships, and preserve interpretability.

To ensure analytical accuracy and reliability, three complementary models were implemented, namely, XGBoost, LightGBM, and CatBoost. Despite all three being gradient boosting frameworks, each adds distinctive feature handling and architectural optimisations that make them suitable to be incorporated in the design.

LightGBM, Light Gradient Boosted Machine, was chosen as the baseline model for its good balance between efficiency, scalability, and accuracy on structured tabular data. It reduces computation time and memory usage by grouping continuous values into bins using a gradient boosting algorithm based on histograms. It grows trees leaf-wise instead of level-wise, which enables lower loss and faster convergence with fewer iterations. It is suitable for the high-cardinality fields in the dataset because it also natively supports categorical features. It generates a foundation for assessing other boosting frameworks given its design, which allows for quick experimentation while preserving high predictive consistency.

XGboost, Extreme Gradient Boosting, was implemented as an additional benchmark model to compare with LightGBM. Using column sampling, shrinkage, and L1/L2 regularisation, it creates an ensemble of decision trees that effectively reduces overfitting and enhances model generalisation. It remains as one of the most dependable and understandable frameworks for tabular data, despite being a little slower than LightGBM. Model transparency and feature effectiveness verification are further supported by its capacity to offer feature importance rankings and SHAP-based explanations.

CatBoost, Categorical Boosting, was chosen for its solid capability of dealing with categorical features through ordered target statistics encoding, which transforms categorical variables into numerical representations without introducing target leakage and is thus very secure in this regard. Compared to other boosting algorithms, this model completely dispenses with the one-hot encoding process and thus maintains a compact feature space, thereby minimizing the risk of overfitting.

In union, the three models presented uniform good performance and the same alignment in identification. It is demonstrated as a practical solution for credit-default prediction for its ability to discern the complex behavioral patterns in customer-level financial data very well.

4.2. Training set-up/validation

A consistent and reproducible training configuration was used across all tests to guarantee that the evaluation of all the models was both statistically accurate and objective. To remove variation resulting from data partitioning, all the three models, namely, LightGBM, XGBoost, and CatBoost were trained on the same dataset splits, preprocessing pipeline, and random seeds.

Using a hold-out split created with sklearn.model_selection.train_test_split, the dataset was divided into two 80% training and 20% testing. This kept the ratio of default and non-default customers consistent across both sets.

Every model utilized X_train, y_train for training and was evaluated on the same held-out test set using the evaluation hook from the library:

- XGBoost: eval_set=[(X_test, y_test)], eval_metric='area under the curve'.
- LightGBM: eval_set=[(X_test, y_test)] with early stopping activated (e.g., early_stopping(…)) and logging callbacks to stop once validation progress stagnated.
- CatBoost: eval_set=(X_test, y_test) featuring integrated validation monitoring.

GPU acceleration has been activated for all compatible models to shorten training duration and enhance efficiency on extensive feature matrices.

After training, the predictions on X_test were sent to sklearn.metrics.roc_auc_score (ROC-AUC), and a personalized AMEX metric function executed in the notebook (top-4% capture + weighted Gini). These functions were applied uniformly across all three models to guarantee a comparable evaluation in the subsequent section.

A fixed random_state = 42 was applied for the train/test division and model creators to ensure consistency in runs. Library seeds remained uniform throughout the experiments. No cross-validation occurred during the training phase since the input table was aggregated to a single row per customer_If, preventing per-customer leakage in a random hold-out split and maintaining manageable runtime.

5. Experimental Study

5.1. Experimentation and Comparison of models

Once the model architectures were determined, every algorithm was executed and assessed via a systematic experimentation process.
The procedure was created to guarantee consistency, equity and reproducibility among all models while precisely evaluating their effectiveness in the credit-default prediction task.

STEP 1: Divide data into training testing sets

A consistent, reproducible hold-out split was employed across all experiments to ensure comparability.
The dataset was separated into 80% for training and 20% for testing subsets through a stratified split to maintain the class distribution of default and non-default customers. A constant random_state = 42 was used to ensure consistent results in different executions. Since each client was present only a single time in the combined dataset, this division was adequate to avoid data leakage and mimic actual generalization performance.

STEP 2: Model Training Process

Every model - LightGBM, XGBoost, and CatBoost - was trained using the identical training subset (x_train, y_train) and assessed on the same validation subset (x_test, y_test).

Throughout the training process, the model's progress was tracked with the corresponding library's evaluation functions to assess the performance and avoid overfitting.

- LightGBM: Trained using a validation set (x_test, y_test) with early stopping implemented after 200 iterations without any enhancement.
- XGBoost: Employed the identical evaluation set with eval_metric+'auc' and a cap of 500 boosting iterations, utilizing regularization and feature sampling to improve generalization.
- CatBoost: Leveraged its built-in eval_set parameter for validation, automatically managing categorical variables and missing data without the need for manual preprocessing.

All models were trained with uniform hyperparameters for learning rate, tree depth, and subsampling ratio to maintain consistency in comparison. GPU acceleration was activated wherever feasible to enhance runtime and memory efficiency.

STEP 3: Assessment and Metric Computation

Post-training, predictions were produced on the reserved test set by utilizing the predict_proba() function to acquire probability scores. 2 assessment metrics were calculated to evaluate performance:

1. ROC-AUC (Receiver Operating Characteristic - Area Under Curve): Assessed the model's capacity to differentiate between default and non-default scenarios at every probability threshold.
2. AMEX metric: Created as a specific function following the official definition from the American Express competition, merging the Top 4% Default Capture Rate with the Weighted Gini Coefficient. This metric highlights ranking

effectiveness and real business worth by rewarding models that accurately pinpoint the most at-risk customers.

STEP 4: Validation and Comparison of Models
A bar chart was generated to visually compare the performance of the 3 gradient-boosting models (LightGBM, XGBoost, and CatBoost) by summarizing the 2 key evaluation metrics - AUC and the AMEX metric.

Following the acquisition of each model's predicted probabilities for the test set, the AUC and AMEX metrics were computed with custom functions defined in Python. The results were organized in a structured dictionary and transformed into a Pandas DataFrame, which was used as the input for plotting.
With Matplotlib, 2 adjacent bars were created for each model: one depicting the AUC score and the other showing the AMEX score. The bar chart distinctly showcased how every model fared on both metrics within one visual representation.
The visualization indicated that LightBGM attained the highest AUC (.9649) and AMEX(0.7947) scores, with XGBoost and CatBoost trailing closely behind, yielding nearly identical outcomes.
This verified that all 3 models showed robust generalization and consistent validation performance, with LightGBM being the most efficient and well-rounded performer overall.

5.2 Validation strategies
A consistent and transparent validation framework was created to guarantee that the performance of each model was assessed fairly and accurately. The identical dataset division, assessment metrics, and testing methods were utilized for all models to facilitate a direct, comparable evaluation of their prediction performance and generalization capability.

5.2.1 Hold-Out Validation Approach
Given that the dataset was previously aggregated at the customer level, a single stratified hold=out validation approach was utilized. The dataset was slit into 80% training and 20% testing subsets utilizing the train_test_split() function with a determined random seed (random_state=42).

Stratification guaranteed that the proportion of default and non-default clients stayed uniform in both groups, avoiding bias from class imbalance. This method enabled the assessment of models on entirely new customers while ensuring computational efficiency which is a crucial factor due to the dataset's vastness.

Since each customer is represented only once in the combined dataset, this approach significantly reduces the chance of data leakage, rendering it appropriate for credit risk modeling on an individual customer basis.

5.2.2 Internal Validation During Training
Every gradient boosting model included its unique internal validation system to track performance throughout training:

- LightGBM utilized an internal evaluation set (eval_set=[(X_test, y_test)]) combined with early stopping to cease training when there was no performance improvement for a defined number of rounds.

- XGBoost also monitored validation performance using eval_metric='auc' and recorded updates at consistent intervals
- CatBoost utilized its built-in validation monitoring through the eval_set parameter, automatically managing overfitting prevention with its ordered boosting technique.

These validation hooks offered immediate insights into the learning progress of each model and made certain that the boosting iterations were maximized while avoiding overfitting.

5.2.3 Evaluation Metrics
To measure predictive performance, two complementary metrics were applied uniformly across all models:

1. ROC-AUC
   The AUC evaluates the model's effectiveness in accurately ranking customers who default compared to those who do not. An AUC value nearer to 1.0 suggests that the model successfully differentiates between the two classes. The precision of ranking is prioritized over accurately predicting probabilities.

2. AMEX Metric
   The AMEX Metric was established as a tailored assessment function derived from the official equation used in the American Express Kaggle contest.

   It joins two elements:
   - Top 4% Default Capture Rate: assesses the model's efficiency in pinpointing the riskiest 4% of clients.
   - Weighted Gini Coefficient: assesses the overall quality of ranking predictions, emphasizing the non-default class more significantly to account for the business impact of misclassification.

   This integrated metric offers a practical evaluation of model effectiveness, prioritizing the correct ordering of high-risk clients rather than focusing solely on accuracy. It offered support to eliminate implementation bias.

Model	Hyperparameter	ROC-AUC	AMEX
LightGBM	n_estimators=5000, learning_rate=0.03, max_depth=-1, num_leaves=255, subsample=0.8, colsample_bytree=0.7, reg_lambda=2.0, objective="binary", n_jobs=-1	.96228	.79431
XGBoost	n_estimators=500, learning_rate=0.05,	.96171	.79268

	max_depth=8, subsample=0.8, colsample_bytree=0.8, eval_metric='auc', tree_method='hist', random_state=42		
CatBoost	iterations=500, learning_rate=0.05, depth=8, loss_function='Logloss', eval_metric='AUC', cat_features=cat_cols, verbose=100	.96181	.79262

6. Fine-tuning

Fine-tuning is an essential step in optimising machine learning model performance. Each gradient boosting model has multiple hyperparameters controlling learning rate, tree depth, subsampling ratios and regularisation strength.

These parameters heavily influence how well the model generalises to unseen data. Fine-tuning helps to identify the most suitable configuration for each model, improving predictive accuracy and robustness.

Instead of manually experimenting with parameters or using exhaustive grid search, this project used Optuna which is an efficient and automated hyperparameter optimisation framework. Optuna was chosen because it employs a Tree-structured Parzen Estimator (TPE) sampler which models the search space probabilistically. This allows it to focus on promising regions and converge faster to near-optimal solutions. It also integrates easily with scikit-learn-like workflows. Compared to random or grid search, Optuna offers higher efficiency and flexibility when tuning multiple models.

Fine-tuning was carried out individually for LightGBM, XGBoost and CatBoost using the same tuning strategy. Each Optuna trial trained a model using a specific set of hyperparameters and evaluated it using AUC. Each model had its own set of hyperparameters to explore and each trial used 3-fold StratifiedKFold cross-validation to evaluate the model's performance. This ensured stable results and reduced the risk of overfitting. After fine-tuning, the best parameters and the corresponding scores were saved as checkpoints for each model.

The following improvements were observed after optimisation:

Model	Baseline AMEX	Tuned AMEX	Improvement
LightGBM	0.794310	0.795962	+0.00165
XGBoost	0.792682	0.795224	+0.00254
CatBoost	0.792617	0.794574	+0.00196

The fine-tuned models showed consistent gains in performance, indicating that the Optuna-driven parameter search successfully identified more optimal configurations for each algorithm. With LightGBM being the best followed by XGBoost and then CatBoost.

After obtaining the best configurations for each individual model, their predictions were combined using the average approach. It was found that the weighted ensemble approach using AMEX score as weights gets the same results as the average approach since the AMEX scores were very close to each other. Hence, the average approach was used to leverage model diversity to reduce variance and achieve better generalisation. This resulted in an AMEX score of 0.796716 on the validation dataset which shows a +0.00241 total improvement from the baseline LightGBM model.

7. Conclusion

Our project has been an insightful and technically enriching experience that deepened our understanding of both machine learning theory and its practical implementation on large-scale financial datasets. In doing this competition, we learned how to handle real-world data challenges such as missing values, imbalanced targets, and high-dimensional features.

From a technical perspective, we gained hands-on experience in:

- Designing end-to-end ML pipelines that include aggregation, preprocessing, feature engineering, and evaluation.
- Implementing and tuning gradient boosting models such as LightGBM, XGBoost and CatBoost, and understanding their strengths in data with non-linear relationship
- Applying GPU acceleration, hyperparameter fine-tuning and evaluation metrics to optimise model performance.

From a teamwork perspective, our ability to collaborate effectively through task allocation and clear communication was strengthened. This was critical in ensuring that our approaches aligned, and to ensure consistency in data preprocessing, model configuration and evaluation across all members.

While we achieved the top 65% ranking on the Kaggle public leaderboard, there remain several areas of improvements. Despite fine-tuning and ensembling, model performance plateaued around an AMEX score of 0.7967. This suggests that the current approach may have reached the limits of what gradient boosting methods can achieve given the dataset's inherent noise and complexity.The improvements gained from hyperparameter tuning were incremental, indicating that further performance gains would likely require deeper feature engineering or more diverse model architectures.

Computational resources and time constraints also posed challenges. GPU tuning and Optuna trials required significant runtime which limited the number of parameter search iterations and the exploration of more complex ensemble strategies. This restriction may have prevented the discovery of higher-performing configurations or alternative model combinations.

For future improvements, the project could explore more advanced feature engineering strategies such as domain-informed aggregations, feature interactions or embedding-based representations. Additionally, more diverse models could be integrated for example deep learning or hybrid architectures that combine sequential and tabular learning to help capture non-linear dependencies that boosting might miss. Further optimization can also be achieved by implementing Optuna's pruning and early stopping features to efficiently search larger parameter spaces within compute budgets. Finally, experiment with more sophisticated ensembling techniques like stacking or blending that can better leverage the strength of different models to achieve stronger overall performance.